

HW_2

이규환 2026406017

카메라 영상에서 중앙의 흰 선과 파란 선을 기준으로 주황색 콘과 초록색 콘이 각각 어느 위치에 있는지를 실시간으로 판별하는 프로그램을 Qt GUI로 만들었다. 위치는 4개의 사분면, 두 선 위의 네 경우, 중앙, 그리고 콘이 보이지 않는 경우까지 모두 10가지로 구분하고, 인식된 콘은 바운딩박스 내부만 잘라 별도의 이미지로 만들어 함께 표시한다. 패키지 이름은 cv_hw2, 실행은 `ros2 run cv_hw2 cv_hw2` 이다.

1. OpenCV 개념 정리

1-1. 비전, OpenCV, 그리고 픽셀

비전은 기계에 사람의 시각과 판단 기능을 부여한 기술이다. 이번 과제로 보면 "콘이 선을 기준으로 어느 쪽에 있나"를 사람은 0.1초면 아는데 그 판단을 프로그램이 대신하게 만드는 것이 목표다. OpenCV는 영상처리에 필요한 함수들을 모아놓은 오픈소스 라이브러리라 색 변환·이진화·필터·모폴로지가 전부 함수 하나로 제공된다. 그래서 실제 작업의 대부분은 알고리즘을 직접 짜는 것이 아니라 **어떤 함수를 어떤 순서로 쓸지 설계하는 일**이 된다.

이미지는 사람 눈에는 사진이지만 컴퓨터에게는 숫자가 격자로 늘어선 배열이다. 격자 한 칸이 픽셀이고 가로 × 세로 픽셀 수가 해상도다. 이번에 쓴 640 × 480 컬러 영상은 픽셀이 307,200개, 픽셀당 값이 3개씩이니 한 프레임이 92만 개가 넘는 숫자를 들고 있다. 이걸 초당 30번 처리해야 실시간이 된다는 뜻이라 나중에 필터 커널 크기를 정할 때 연산량을 신경 쓰게 됐다. OpenCV에서 이 배열을 담는 자료형이 `cv::Mat` 이다.

1-2. 색을 표현하는 네 가지 방식

표현 방식	채널 수	값의 범위	픽셀 하나가 들고 있는 것
RGB	3	각 0 ~ 255	빨강, 초록, 파랑의 세기
Grayscale	1	0 ~ 255	밝기 하나
HSV	3	H 0~179, S·V 0~255	색조, 채도, 명도
Binary	1	0 또는 255	검정이나 흰색이나

Grayscale은 정보량이 RGB의 3분의 1이라 연산 속도에서 이득을 보지만, 이번 과제는 주황과 초록을 구분해야 하므로 색 정보를 버릴 수 없었다. 처음에 헛갈렸던 것이 두 가지 있다. **OpenCV의 채널 순서는 RGB가 아니라 BGR이고, HSV의 H는 0~255가 아니라 0~179다.** 색상환이 원래 0~360도인데 한 바이트에 넣느라 절반으로 줄여 쓰기 때문에 H값 1이 실제로는 2도에 해당한다.

1-3. 왜 RGB 가 아니라 HSV 인가

처음에는 "주황콘은 R값이 크고 G값이 중간인 픽셀"이라고 단순하게 생각했는데 이게 안 된다. 조명이 밝아지면 R, G, B가 전부 같이 올라가고 그늘이 지면 전부 같이 내려가서, RGB에서는 "무슨 색인가"와 "얼마나 밝은가"를 떼어낼 수 없기 때문이다. HSV 는 이 문제를 풀려고 만든 표현 방식이다.

- **H (색조)** — 색상환 위의 각도. 조명이 밝아져도 잘 변하지 않는다.
- **S (채도)** — 색이 얼마나 진한지. 흰색·회색·검은색은 0 에 가깝다.
- **V (명도)** — 얼마나 밝은지. 조명의 영향을 이 채널이 대부분 받아준다.

그래서 색 추출의 기본 전략이 정해진다. **H로 색 계열을 고르고 S와 V로 너무 흐리거나 어두운 것을 걸러낸다.** 이번처럼 네 대상을 동시에 구분해야 할 때, 대상마다 H만 다르게 주고 S·V는 비슷하게 가져갈 수 있다는 것이 큰 장점이었다.

1-4. 이진화와 inRange

이진화는 픽셀을 검은색(0)과 흰색(255) 두 가지로만 표현하는 것이다. 찾으려는 대상

만 흰색으로 남기면 이후 처리는 "흰 영역이 어디에 얼마나 있는가"만 보면 되므로 훨씬 단순해진다. OpenCV에서는 inRange로 특정 색 범위만 뽑아 이진화한다.

```
Mat img_hsv;
cvtColor(img, img_hsv, COLOR_BGR2HSV);
inRange(img_hsv, Scalar(h_low, s_low, v_low),
        Scalar(h_high, s_high, v_high), mask);
```

여기서 가장 중요한 건 세 조건이 **AND로 묶인다**는 점이다. H가 아무리 정확해도 S가 하한보다 낮으면 그 픽셀은 버려진다. "H는 맞췄는데 화면이 전부 까맣다"는 상황의 범인은 거의 항상 S나 V의 하한이었다.

1-5. 가우시안 필터

어떤 픽셀이 주변과 극단적으로 값이 다르면 그게 노이즈다. 이진화처럼 값을 딱 잘라내는 연산 앞에서는 이 흔들림을 미리 눌러줘야 한다. 가우시안 필터는 가운데에 가중치를 크게 주고 멀어질수록 작게 주는 커널로 주변 값을 섞는다. 가중치 합이 1이 되도록 나누기 때문에 전체 밝기는 유지되면서 튀는 값만 완만해진다. ksize는 중심 픽셀이 있어야 하므로 반드시 홀수여야 한다.

```
GaussianBlur(src, dst, Size(5, 5), 0);
```

HW1에서 이 필터의 효과를 수치로 확인했는데, 공 사진의 흰색 덩어리 개수가 필터 없이 17개에서 5×5 적용 후 4개로 줄었다. 작은 노이즈 점만 골라 사라지고 물체는 그대로 남는다는 것을 그때 확인했기 때문에, 이번에는 5×5 를 고정으로 항상 적용하도록 만들었다.

1-6. 모폴로지

모폴로지는 이진 이미지의 구조와 형태를 다루는 연산이다. 가우시안 필터가 이진화 **전에** 원본을 다듬는 것이라면 모폴로지는 이진화 **후에** 결과를 다듬는다는 점에서 역할이 다르다.

연산	동작	효과
침식 (erode)	객체 외곽을 깎아낸다	객체가 작아지고 작은 점이 사라진다

연산	동작	효과
팽창 (dilate)	객체 외곽을 확장한다	객체가 커지고 작은 구멍이 메워진다
열기 (open)	침식 후 팽창	작은 노이즈 제거에 효과적
닫기 (close)	팽창 후 침식	내부의 빈 구멍을 없애는 데 효과적

열기가 왜 그런 효과를 내는지 처음엔 와닿지 않았다. 침식으로 작은 점을 아예 지운 다음 팽창으로 남은 객체 크기만 원래대로 되돌리는 것이다. 점은 이미 사라졌으니 되살아나지 않는다. 닫기는 반대로 팽창으로 구멍을 먼저 메우고 침식으로 크기를 되돌리므로 메워진 구멍이 그대로 남는다.

```
Mat kernel = getStructuringElement(MORPH_RECT, Size(5, 5));
morphologyEx(mask, mask, MORPH_OPEN, kernel);
morphologyEx(mask, mask, MORPH_CLOSE, kernel);
```

이번 과제에서는 **열기 다음에 닫기** 순서로 적용했다. 먼저 열기로 배경에 흩어진 점 노이즈를 지우고, 닫기로 콘 표면의 반사광 때문에 뚫린 구멍을 메우기 위해서다. 순서를 반대로 하면 닫기가 노이즈 점들을 이어붙여 더 큰 덩어리를 만들 수 있어서 이 순서가 맞다고 판단했다.

2. 과제 분석과 설계

2-1. 판별해야 하는 10 가지 경우

구분	경우	판정 조건
사분면	1 / 2 / 3 / 4 사분면	흰선 중심과 파란선 중심 기준 상하좌우
흰선 위	위쪽 / 아래쪽 하얀선 위	콘 중심 x가 흰선의 x 범위 안
파란선 위	왼쪽 / 오른쪽 파란선 위	콘 중심 y가 파란선의 y 범위 안
중앙	중앙	두 조건을 동시에 만족
없음	존재하지 않음	콘 마스크의 면적이 기준 미만

설계상 중요한 판단이 하나 있었다. "선 위"인지 "사분면"인지를 가르려면 선을 **면적**

을 가진 띠로 봐야 한다는 점이다. 선을 좌표 하나로만 보면 콘 중심이 정확히 그 좌표에 놓일 확률이 거의 0이라 "선 위"가 영영 나오지 않는다. 그래서 선의 바운딩박스가 가진 폭과 높이를 그대로 판정 범위로 쓰기로 했다.

2-2. 전체 파이프라인

```
카메라 프레임 -> GaussianBlur 5x5 -> cvtColor(BGR2HSV)
-> inRange x 4 (흰선, 파란선, 초록콘, 주황콘)
-> morphologyEx (열기 -> 닫기)
-> countNonZero 로 존재 여부, boundingRect 로 위치
-> 위치 판별 + 화면 출력 + 바운딩박스 크롭
```

네 대상이 완전히 같은 경로를 지나고 HSV 범위만 다르다. 그래서 마스크 생성을 makeMask 함수 하나로 묶고 대상 번호만 인자로 받아 반복문으로 네 번 돌리는 구조로 만들었다.

2-3. findContours 를 쓰지 않은 이유

물체 위치를 찾는다고 하면 보통 findContours로 외곽선을 찾아 가장 큰 것을 고른다. 그런데 이 함수는 교육자료에서 다루지 않은 내용이라 배운 범위 안에서 풀 방법을 찾아봤다.

답은 boundingRect에 있었다. 이 함수는 **1채널 이진 이미지를 그대로 넣으면 0이 아닌 픽셀 전체를 감싸는 사각형**을 돌려준다. 이진화 결과에 노이즈가 없다면 그게 곧 물체의 바운딩박스가 된다. 존재 여부는 countNonZero로 흰 픽셀을 세서 기준값(300 픽셀)과 비교하면 된다.

```
cv::Rect MainWindow::findObject(const cv::Mat& mask)
{
    if (cv::countNonZero(mask) < MIN_AREA)
    {
        return cv::Rect();
    }
    return cv::boundingRect(mask);
}
```

다만 이 방법에는 분명한 약점이 있다. boundingRect는 흰 픽셀 **전부**를 감싸므로 화면 구석에 노이즈 점 하나만 남아도 박스가 화면 전체 크기로 커져버린다. findContours였다면 가장 큰 덩어리만 고르면 되니 이런 문제가 없다. 그래서 이 설계

에서는 **이진화 결과가 깨끗한 것이 선택이 아니라 필수 조건**이 된다. 가우시안 필터와 모폴로지 열기를 반드시 거치게 만든 이유, 그리고 4장에서 HSV 값을 오래 붙잡고 있었던 이유가 전부 여기서 나온다. 배운 범위로 제약을 두니 오히려 전처리의 중요성을 더 크게 체감하게 됐다.

2-4. 기준선을 잡는 방법

흰 선은 세로로 긴 물체이므로 바운딩박스의 x 범위와 중심 x 가 의미를 가지고, 파란 선은 가로로 길어서 y 범위와 중심 y 를 쓴다. 선 검출이라고 하면 허프 변환을 떠올리기 쉽지만 선이 세로 또는 가로로 고정된 이번 경우에는 바운딩박스만으로 충분했다. 실제로 파란 선은 중간을 흰 선이 가로지르고 콘이 일부를 가려 마스크가 서너 조각으로 끊겨 나왔는데, `boundingRect`는 조각 전체를 감싸므로 끊긴 것과 상관없이 올바른 y 범위가 나온다. 조각을 이어붙이는 처리를 따로 하지 않아도 되는 점이 이 방법의 장점이었다.

2-5. 위치 판별 로직

콘의 중심 좌표를 구하고, 그 좌표가 흰 선의 x 범위 안에 있는지(`on_white`)와 파란 선의 y 범위 안에 있는지(`on_blue`)를 먼저 따진다. 이 두 참/거짓 조합으로 대부분이 갈린다.

on_white	on_blue	추가 조건	결과
O	O	-	중앙
O	X	콘 $y <$ 파란선 중심 y	위쪽 하얀선 위
O	X	콘 $y >$ 파란선 중심 y	아래쪽 하얀선 위
X	O	콘 $x <$ 흰선 중심 x	왼쪽 파란선 위
X	O	콘 $x >$ 흰선 중심 x	오른쪽 파란선 위
X	X	오른쪽 + 위쪽 / 왼쪽 + 위쪽	1사분면 / 2사분면
X	X	왼쪽 + 아래쪽 / 오른쪽 + 아래쪽	3사분면 / 4사분면

여기에 예외 두 가지를 앞에 붙였다. 콘의 바운딩박스가 비어 있으면 "존재하지 않음"을, 기준선이 검출되지 않았으면 "기준선 없음"을 반환한다. 두 번째를 넣은 이유는 기준선이 없을 때 중심 좌표가 0이 되어 모든 콘이 1사분면이나 4사분면으로 잘못 판정되는 것을 실제로 겪었기 때문이다. 틀린 답을 내놓는 것보다 판단을 보류하는 편이 낫다고 생각했다.

2-6. 바운딩박스 내부 추출

콘이 인식되면 원본 프레임에서 바운딩박스 영역만 잘라 새 이미지로 만든다. 여기서 **clone()**을 반드시 붙여야 한다. clone 없이 받으면 원본 프레임의 메모리를 그대로 공유해서, 다음 프레임이 들어오는 순간 잘라낸 이미지의 내용도 같이 바뀌어 버린다. 정지 화상이라면 모르고 넘어갔을 텐데 실시간이라 화면이 깜빡이는 것으로 바로 드러났다.

```
crop_neon_ = frame_(neon_rect).clone();
```

3. 구현

3-1. GUI 구성

교육자료의 예시 화면을 참고해서 처리 단계가 한눈에 보이도록 화면을 6분할했다. 원본(Usb_Cam), 기준선과 바운딩박스를 그려 넣은 결과(Find_object), 그리고 네 대상의 이진화 결과를 각각 띄운다. 중간 결과를 전부 보여주는 이유는 HSV 값을 맞출 때 어느 단계에서 잘못됐는지 눈으로 바로 확인하기 위해서다. 아래쪽에는 조절 대상을 고르는 라디오 버튼 네 개, H·S·V의 하한과 상한 슬라이더 여섯 개, 판별 결과와 잘라낸 콘 이미지, 카메라 선택 줄을 배치했다. Find_object 화면에는 흰 선을 노란색 세로선, 파란 선을 자홍색 가로선, 콘을 초록색과 주황색 사각형으로 그려서 프로그램이 무엇을 어디로 인식했는지 드러나게 했다.

3-2. 네 대상의 HSV 값을 따로 관리하기

슬라이더는 여섯 개인데 조절할 대상은 네 개다. 그래서 값을 2차원 배열에 담고 라디오 버튼으로 대상을 바꿀 때마다 해당 값을 슬라이더에 불러오게 했다. 슬라이더를 움직일 때마다 현재 대상의 배열에 곧바로 저장해두면 대상을 바꿀 때는 불러오기만 하면 되므로 값이 날아갈 일이 없다. 다만 코드로 슬라이더 값을 바꾸면 `valueChanged` 시그널이 또 발생해 저장 함수가 불리고 그게 다시 화면 갱신을 부르는 문제가 있었다. `bool` 변수 하나를 두고 코드로 값을 넣는 동안만 켜서 그 사이의 시그널은 무시하도록 처리했다.

3-3. 카메라 입력

처음에는 이미지 파일과 카메라를 전환하는 방식으로 만들었다가 실시간 확인이 과제 목적에 맞아 카메라 전용으로 바꿨다. `/dev` 폴더에서 `video`로 시작하는 장치를 전부 찾은 다음 각각을 `VideoCapture`로 한 번 열어보고 **실제로 열리는 것만** 목록에 넣도록 했다. 리눅스에서는 카메라 하나에 `video` 노드가 두 개씩 생기는 경우가 있는데 그중 하나는 영상 캡처용이 아니어서, 번호만 나열하면 열리지 않는 장치를 고르게 되기 때문이다. 프레임은 `QTimer`를 33밀리초 주기로 돌려 읽으므로 초당 약 30프레임이 된다.

교육자료의 예시 화면이 `usb_cam` 노드의 토픽을 받는 구조였기 때문에 목록 맨 아래에 ROS 토픽(`/image_raw`)을 고르는 항목도 남겨뒀다. 이때 Image 메시지를 `cv::Mat`으로 바꿔야 하는데 `cv_bridge` 없이 직접 변환했다. Image 메시지가 사실상 바이트 배열에 크기와 인코딩 정보가 붙은 형태라 Mat 생성자에 그대로 넘기면 되기 때문이다. 인코딩이 `rgb8`이면 채널 순서가 반대이므로 `cvtColor`로 바꿔준다.

```
cv::Mat received(msg->height, msg->width, CV_8UC3,
    msg->data.data(), msg->step);
```

3-4. ROS 스레드와 Qt 스레드

Qt의 `exec()`와 ROS의 `spin()`은 둘 다 끝나지 않는 루프라서 한 프로그램에서 그냥 같이 돌릴 수 없다. 그래서 ROS 쪽은 `QThread`를 상속한 `QNode`의 `run()` 안에서 `spin_some()`으로 돌린다. 여기서 주의할 점은 ROS 콜백이 **Qt 메인 스레드가 아닌 별**

도 스레드에서 실행된다는 것이다. 콜백에서 위젯을 직접 건드리면 화면이 깨지거나 프로그램이 죽는다. 그래서 콜백에서는 뮤텍스로 잠그고 프레임을 멤버 변수에 저장한 뒤 시그널만 보내고 끝내며, 실제 화면 갱신은 그 시그널을 받은 메인 스레드에서 이루어진다.

4. HSV 값 튜닝 과정

이번 과제에서 시간을 가장 많이 쓴 부분이 여기다. 코드를 다 짜고 나서도 콘이 잡히지 않아서 화면의 픽셀값을 직접 찍어보며 원인을 하나씩 찾아야 했다.

4-1. 과노출 — 물체의 중심이 하얗게 날아간다

처음 기본값은 콘이 선명한 색이라고 가정하고 채도 하한을 높게(초록콘 90, 주황콘 150) 잡았는데, 실제 카메라 영상에서는 두 콘 모두 이진화 결과가 거의 검은 화면으로 나왔다. 픽셀값을 재보니 카메라 노출이 세서 **콘의 중심부가 하얗게 날아가 있었다**. 초록콘 몸통의 채도 중앙값이 35, 주황콘이 116이었다. 색이 선명한 부분은 얇은 테두리뿐이고 안쪽은 흰색에 가까웠던 것이다. 하한을 90, 150으로 잡아줬으니 몸통이 통째로 걸러지고 테두리만 남았는데 그마저 모폴로지 열기에 지워져 아무것도 남지 않았다. 채도 하한을 초록콘 20, 주황콘 70으로 크게 낮추자 콘 전체가 잡히기 시작했다. **물체가 원래 무슨 색인지가 아니라 카메라에 찍힌 값이 얼마인지를 봐야 한다는 것**이 여기서 얻은 교훈이다.

4-2. 흰 선과 형광 초록콘이 겹치는 문제

과노출은 흰 선 쪽에도 문제를 만들었다. 흰 선은 명도가 높고 채도가 낮은 것으로 찾는데, 하얗게 날아간 초록콘 중심부도 똑같이 명도가 높고 채도가 낮았다. 그래서 White_line 화면에 선과 함께 초록콘 덩어리가 잡혔고, 앞서 말한 boundingRect의 약점 때문에 기준선의 x 범위가 콘 쪽까지 늘어나면서 콘이 "아래쪽 하얀선 위"로 잘못 판정됐다. 두 영역의 픽셀값을 비교해보니 갈라지는 지점이 있었다. **콘은 아무리 날아가도 초록빛이 남아 채도가 0보다 크고, 흰 선만 정확히 0이었다**. 흰색은 세 채널이 같

은 값이라 채도가 정의상 0이 되지만 초록콘은 초록 채널이 먼저 포화되어 나머지와 차이가 남기 때문이다. 채도 상한을 0으로 조이자 흰 선만 남았다.

4-3. 파란 선과 캐비닛, 주황콘과 나무 선반

배경에 파란 회색 캐비닛이 있었는데 색조를 재보니 파란 선이 101, 캐비닛도 101 근처로 거의 같아서 색조로는 구분할 수 없었다. 나머지 두 채널을 보니 채도는 선이 94~124, 캐비닛이 67~78이고 명도는 선이 220 근처까지 올라가는 데 비해 캐비닛은 최대 185에서 끝났다. 채도 하한 90과 명도 하한 190을 같이 걸자 캐비닛이 완전히 걸러졌다.

화면 오른쪽 위의 나무 선반은 주황콘과 가장 비슷했다. 색조는 선반 19, 콘 22로 겹치고 채도도 선반 68, 콘 79로 거의 차이가 없어서 채도 하한을 올리면 선반과 함께 콘도 사라졌다. 여기서도 명도가 답이었다. 선반 픽셀의 명도는 98퍼센트 지점이 225에서 끝나는데 주황콘은 과노출 덕분에 251까지 올라간다. 명도 하한을 225로 두자 선반은 한 픽셀도 남지 않고 콘만 깔끔하게 잡혔다.

4-4. 최종 기본값

대상	Hue	Saturation	Value	핵심 판단 근거
White_line	0 ~ 179	0 ~ 0	250 ~ 255	채도가 정확히 0인 것만
Blue_line	90 ~ 115	90 ~ 255	190 ~ 255	명도로 캐비닛 제외
Neon_cone	28 ~ 55	20 ~ 255	150 ~ 255	채도 하한을 크게 낮춤
Orange_cone	0 ~ 30	70 ~ 255	225 ~ 255	명도로 나무 선반 제외

네 대상을 맞추면서 공통적으로 배운 것이 있다. 두 물체가 헛갈릴 때는 세 채널을 전부 조이는 것이 아니라 **세 채널 중 가장 잘 갈라지는 하나를 찾아 거기에 기준을 거는**

것이 훨씬 효과적이라는 점이다. 색조로 겹치면 채도를, 채도로도 겹치면 명도를 보는 식이다. HSV가 세 채널로 나뉘어 있는 이유를 이 과정에서 몸으로 이해하게 됐다. 다만 이 값들은 지금의 조명과 카메라 설정에 맞춘 것이라 환경이 바뀌면 다시 잡아야 하므로, 슬라이더를 GUI에 남겨두고 reset slider 버튼으로 언제든지 기본값으로 되돌릴 수 있게 했다.

5. 실행 결과

카메라를 선택하면 바로 영상이 들어오고 여섯 개 화면이 초당 30프레임으로 동시에 갱신된다. Find_object 화면에 노란 세로선과 자홍 가로선이 기준선 위치에 그려지고 두 콘에 각각 초록색과 주황색 사각형이 표시되며, 오른쪽 Position 영역에 두 콘의 판별 결과가, 그 아래에 잘라낸 콘 이미지가 실시간으로 나온다. 콘을 손으로 옮기면 판별 결과가 프레임마다 바로 따라 바뀌고, 화면 밖으로 빠지면 "존재하지 않음"으로 바뀌면서 잘라낸 이미지 칸도 비워진다. 콘을 선 쪽으로 천천히 밀면 콘 중심이 선의 폭 안으로 들어가는 순간 사분면 표시가 "선 위"로 바뀌고, 두 선이 만나는 지점에서는 "중앙"이 표시된다. 선을 좌표 하나가 아니라 띠로 본 설계가 의도대로 동작하는 것을 여기서 확인할 수 있었다.

전체 실행 영상은 같은 폴더에 첨부했다. 프로그램을 실행해 카메라를 선택하는 것부터, 콘을 여러 위치로 옮겨가며 판별 결과가 실시간으로 바뀌는 모습, 슬라이더로 HSV 값을 조절할 때 이진화 결과가 변하는 모습까지 담았다.

6. 정리하며

- HSV 는 색과 밝기가 채널로 분리되어 있어 조명이 일정하지 않은 환경에서 특정 색을 찾을 때 RGB 보다 유리하다.
- inRange 의 세 조건은 AND 로 묶이므로 색이 안 잡힐 때는 색조보다 채도와 명도의 하한을 먼저 의심해야 한다.

- boundingRect 로 물체 위치를 잡는 방법은 간단하지만 노이즈에 취약해서 전처리가 필수가 된다.
- 두 물체가 헛갈릴 때는 세 채널을 모두 조이는 대신 가장 잘 갈라지는 채널 하나를 찾아 기준을 거는 것이 효과적이다.
- 실시간 영상에서 잘라낸 이미지는 반드시 clone 으로 복사해야 다음 프레임에 내용이 바뀌지 않는다.

가장 크게 남은 것은 튜닝 과정이었다. 처음에는 HSV 값을 맞추는 일이 감으로 슬라이더를 움직이는 단순 작업이라고 생각했는데, 실제로는 구분하려는 두 대상의 픽셀 값 분포를 보고 어느 채널에서 갈라지는지를 찾는 분석 작업이었다. 값이 안 맞을 때 무작정 슬라이더를 흔드는 대신 원인을 좁혀가는 방법을 익힌 것이 이번 과제의 가장 큰 수확이라고 생각한다.